flute v1.8.1

File Delivery over Unidirectional Transport (FLUTE)

#network #broadcast #multicast #satellite #5g

Follow

FLUTE - File Delivery over Unidirectional Transport

Massively scalable multicast distribution solution

The library implements a unidirectional file delivery, without the need of a return channel.

RFC

This library implements the following RFCs

RFC	Title	Link
RFC 6726	FLUTE - File Delivery over Unidirectional Transport	https://www.rfc-editor.org/rfc/rfc6726.html
RFC 5775	Asynchronous Layered Coding (ALC) Protocol Instantiation	https://www.rfc-editor.org/rfc/rfc5775.html
RFC 5661	Layered Coding Transport (LCT) Building Block	https://www.rfc-editor.org/rfc/rfc5651
RFC 5052	Forward Error Correction (FEC) Building Block	https://www.rfc-editor.org/rfc/rfc5052
RFC 5510	Reed-Solomon Forward Error Correction (FEC) Schemes	https://www.rfc-editor.org/rfc/rfc5510.html

RFC	Title	Link
3GPP TS 26.346	Extended FLUTE FDT Schema (7.2.10)	https://www.etsi.org/deliver/etsi_ts/126300_126399/126346/17.03.00_60/ts_126346v17(

Thread Safety

FLUTE Sender

The FLUTE Sender is designed to be safely shared between multiple threads.

FLUTE Receiver and Tokio Integration

Unlike the sender, the FLUTE Receiver **is not thread-safe** and cannot be shared between multiple threads. To integrate it with Tokio, you must use `tokio::task::LocalSet`, which allows spawning tasks that require a single-threaded runtime.

The following example demonstrates how to use the FLUTE Receiver with Tokio:

```
use flute::receiver::{writer, MultiReceiver};
use std::rc::Rc;

#[tokio::main]
async fn main() {
   let local = task::LocalSet::new();
   // Run the local task set.
   local.run_until(async move {
       let nonsend_data = nonsend_data.clone();
       task::spawn_local(async move {
           let writer = Rc::new(writer::ObjectWriterFSBuilder::new(&std::path::Path::new("./flute_dir")
           let mut receiver = MultiReceiver::new(writer, None, false);
           // ... run the receiver
       }).await.unwrap();
   }).await;
}
```

UDP/IP Multicast files sender

Transfer files over a UDP/IP network

```
use flute::sender::Sender;
use flute::sender::ObjectDesc;
use flute::core::lct::Cenc;
use flute::core::UDPEndpoint;
use std::net::UdpSocket;
use std::time::SystemTime;

// Create UDP Socket
let udp_socket = UdpSocket::bind("0.0.0.0:0").unwrap();
udp_socket.connect("224.0.0.1:3400").expect("Connection failed");

// Create FLUTE Sender
let tsi = 1;
let oti = Default::default();
let config = Default::default();
let endpoint = UDPEndpoint::new(None, "224.0.0.1".to_string(), 3400);
let mut sender = Sender::new(endpoint, tsi, &oti, &config);
```

```
// Add object(s) (files) to the FLUTE sender (priority queue 0)
let obj = ObjectDesc::create_from_buffer(b"hello world".to_vec(), "text/plain",
&url::Url::parse("file:///hello.txt").unwrap(), 1, None, None, None, None, Cenc::Null, true, None, true)
sender.add_object(0, obj);

// Always call publish after adding objects
sender.publish(SystemTime::now());

// Send FLUTE packets over UDP/IP
while let Some(pkt) = sender.read(SystemTime::now()) {
    udp_socket.send(&pkt).unwrap();
    std::thread::sleep(std::time::Duration::from_millis(1));
}
```

UDP/IP Multicast files receiver

Receive files from a UDP/IP network

```
use flute::receiver::{writer, MultiReceiver};
use flute::core::UDPEndpoint;
use std::net::UdpSocket;
use std::time::SystemTime;
use std::rc::Rc;

// Create UDP/IP socket to receive FLUTE pkt
let endpoint = UDPEndpoint::new(None, "224.0.0.1".to_string(), 3400);
let udp_socket = UdpSocket::bind(format!("{}:{}", endpoint.destination_group_address, endpoint.port)).ex

// Create a writer able to write received files to the filesystem
let writer = Rc::new(writer::ObjectWriterFSBuilder::new(&std::path::Path::new("./flute_dir"))
    .unwrap_or_else(|_| std::process::exit(0)));

// Create a multi-receiver capable of de-multiplexing several FLUTE sessions
let mut receiver = MultiReceiver::new(writer, None, false);

// Receive pkt from UDP/IP socket and push it to the FLUTE receiver
let mut buf = [0; 2048];
loop {
    let (n, _src) = udp_socket.recv_from(&mut buf).expect("Failed to receive data");
    let now = SystemTime::now();
    receiver.push(&endpoint, &buf[..n], now).unwrap();
    receiver.cleanup(now);
}
```

Application-Level Forward Erasure Correction (AL-FEC)

The following error recovery algorithms are supported

- [x] No-code
- [x] Reed-Solomon GF 2^8
- [x] Reed-Solomon GF 2^8 Under Specified
- [ ] Reed-Solomon GF 2^16
- [ ] Reed-Solomon GF 2^m
- [x] RaptorQ
- [x] Raptor

The `Oti` module provides an implementation of the Object Transmission Information (OTI) used to configure Forward Error Correction (FEC) encoding in the FLUTE protocol.

```
use flute::sender::Sender;
use flute::core::Oti;
use flute::core::UDPEndpoint;

// Reed Solomon 2^8 with encoding blocks composed of
// 60 source symbols and 4 repair symbols of 1424 bytes per symbol
let endpoint = UDPEndpoint::new(None, "224.0.0.1".to_string(), 3400);
let oti = Oti::new_reed_solomon_rs28(1424, 60, 4).unwrap();
let mut sender = Sender::new(endpoint, 1, &oti, &Default::default());
```

Content Encoding (CENC)

The following schemes are supported during the transmission/reception

- ☑ Null (no compression)
- ☑ Deflate
- ☑ Zlib
- ☑ Gzip

Files multiplex / Blocks interleave

The FLUTE Sender is able to transfer multiple files in parallel by interleaving packets from each file. For example:

Pkt file1 -> Pkt file2 -> Pkt file3 -> **Pkt file1** -> Pkt file2 -> Pkt file3 ...

The Sender can interleave blocks within a single file. The following example shows Encoding Symbols (ES) from different blocks (B) are interleaved. For example:

(B 1,ES 1)->(B 2,ES 1)->(B 3,ES 1)->**(B 1,ES 2)**->(B 2,ES 2)...

To configure the multiplexing, use the `Config` struct as follows:

```
use flute::sender::Sender;
use flute::sender::Config;
use flute::sender::PriorityQueue;
use flute::core::UDPEndpoint;

let mut config = Config {
    // Interleave a maximum of 3 blocks within each file
    interleave_blocks: 3,
    ..Default::default()
};

// Interleave a maximum of 3 files in priority queue '0'
config.set_priority_queue(PriorityQueue::HIGHEST, PriorityQueue::new(3));

let endpoint = UDPEndpoint::new(None, "224.0.0.1".to_string(), 3400);
let mut sender = Sender::new(endpoint, 1, &Default::default(), &config);
```

Priority Queues

FLUTE sender can be configured with multiple queues, each having a different priority level. Files in higher priority queues are always transferred before files in lower priority queues. Transfer of files in lower priority queues is paused while there are files to be transferred in higher priority queues.

```
use flute::sender::Sender;
use flute::sender::Config;
use flute::sender::PriorityQueue;
use flute::core::UDPEndpoint;
```

```
use flute::sender::ObjectDesc;
use flute::core::lct::Cenc;

// Create a default configuration
let mut config: flute::sender::Config = Default::default();

// Configure the HIGHEST priority queue with a capacity of 3 simultaneous file transfer
config.set_priority_queue(PriorityQueue::HIGHEST, PriorityQueue::new(3));

// Configure the LOW priority queue with a capacity of 1 file transfer at a time
config.set_priority_queue(PriorityQueue::LOW, PriorityQueue::new(1));

let endpoint = UDPEndpoint::new(None, "224.0.0.1".to_string(), 3400);
let mut sender = Sender::new(endpoint, 1, &Default::default(), &config);

// Create an ObjectDesc for a low priority file
let low_priority_obj = ObjectDesc::create_from_buffer(b"low priority".to_vec(), "text/plain",
&url::Url::parse("file:///low_priority.txt").unwrap(), 1, None, None, None, None, Cenc::Null, true, None
```

Bitrate Control

The FLUTE library does not provide a built-in bitrate control mechanism. Users are responsible for controlling the bitrate by sending packets at a specific rate. However, the library offers a way to control the target transfer duration or the target transfer end time for each file individually.

To ensure proper functionality, the target transfer mechanism requires that the overall bitrate is sufficiently high.

Target Transfer Duration

The sender will attempt to transfer the file within the specified duration.

```
use flute::sender::Sender;
use flute::sender::ObjectDesc;
use flute::sender::TargetAcquisition;
use flute::core::lct::Cenc;
use flute::core::UDPEndpoint;
use std::net::UdpSocket;
use std::time::SystemTime;

// Create UDP Socket
let udp_socket = UdpSocket::bind("0.0.0.0:0").unwrap();
udp_socket.connect("224.0.0.1:3400").expect("Connection failed");

// Create FLUTE Sender
let tsi = 1;
let oti = Default::default();
let config = Default::default();
let endpoint = UDPEndpoint::new(None, "224.0.0.1".to_string(), 3400);
let mut sender = Sender::new(endpoint, tsi, &oti, &config);

// Create an Object
let mut obj = ObjectDesc::create_from_buffer(b"hello world".to_vec(), "text/plain",
```

```
&url::Url::parse("file:///hello.txt").unwrap(), 1, None, None, None, None, Cenc::Null, true, None, true)

// Set the Target Transfer Duration of this object to 2 seconds
obj.target_acquisition = Some(TargetAcquisition::WithinDuration(std::time::Duration::from_secs(2)));

// Add object(s) (files) to the FLUTE sender (priority queue 0)
sender.add_object(0, obj);

// Always call publish after adding objects
sender.publish(SystemTime::now());

// Send FLUTE packets over UDP/IP
while sender.nb_objects() > 0  {
    if let Some(pkt) = sender.read(SystemTime::now()) {
        udp_socket.send(&pkt).unwrap();
    } else {
       std::thread::sleep(std::time::Duration::from_millis(1));
    }
}
```

Target Time to End Transfer

The sender will try to finish the file at the specified time.

```
use flute::sender::Sender;
use flute::sender::ObjectDesc;
use flute::sender::TargetAcquisition;
use flute::core::lct::Cenc;
use flute::core::UDPEndpoint;
use std::net::UdpSocket;
use std::time::SystemTime;

// Create UDP Socket
let udp_socket = UdpSocket::bind("0.0.0.0:0").unwrap();
udp_socket.connect("224.0.0.1:3400").expect("Connection failed");

// Create FLUTE Sender
let tsi = 1;
let oti = Default::default();
let config = Default::default();
let endpoint = UDPEndpoint::new(None, "224.0.0.1".to_string(), 3400);
let mut sender = Sender::new(endpoint, tsi, &oti, &config);

// Create an Object
let mut obj = ObjectDesc::create_from_buffer(b"hello world".to_vec(), "text/plain",
&url::Url::parse("file:///hello.txt").unwrap(), 1, None, None, None, None, Cenc::Null, true, None, true)

// Set the Target Transfer End Time of this object to 10 seconds from now
let target_end_time = SystemTime::now() + std::time::Duration::from_secs(10);
obj.target_acquisition = Some(TargetAcquisition::WithinTime(target_end_time));

// Add object(s) (files) to the FLUTE sender (priority queue 0)
sender.add_object(0, obj);

// Always call publish after adding objects
sender.publish(SystemTime::now());

// Send FLUTE packets over UDP/IP
while sender.nb_objects() > 0  {
    if let Some(pkt) = sender.read(SystemTime::now()) {
        udp_socket.send(&pkt).unwrap();
    } else {
       std::thread::sleep(std::time::Duration::from_millis(1));
```

```
    }
}
```

Python bindings

Installation

```
pip install flute-alc
```

Example

Flute Sender python example

```
from flute import sender

# Flute Sender config parameters
sender_config = sender.Config()

# Object transmission parameters (no_code => no FEC)
# encoding symbol size : 1400 bytes
# Max source block length : 64 encoding symbols
oti = sender.Oti.new_no_code(1400, 64)

# Create FLUTE Sender
flute_sender = sender.Sender(1, oti, sender_config)

# Transfer a file
flute_sender.add_file("/path/to/file", 0, "application/octet-stream", None, None)
flute_sender.publish()

while True:
    alc_pkt = flute_sender.read()
    if alc_pkt == None:
        break

    #TODO Send alc_pkt over UDP/IP
```

Flute Receiver python example

```
from flute import receiver

# Write received objects to a destination folder
receiver_writer = receiver.ObjectWriterBuilder("/path/to/dest")

# FLUTE Receiver configuration parameters
receiver_config = receiver.Config()

tsi = 1

# Create a FLUTE receiver with the specified endpoint, tsi, writer, and configuration
udp_endpoint = receiver.UDPEndpoint("224.0.0.1", 1234)
flute_receiver = receiver.Receiver(udp_endpoint, tsi, receiver_writer, receiver_config)

while True:
    # Receive LCT/ALC packet from UDP/IP multicast (Implement your own receive_from_udp_socket() fur
    # Note: FLUTE does not handle the UDP/IP layer, you need to implement the socket reception mecha
    pkt = receive_from_udp_socket()
```

```
        # Push the received packet to the FLUTE receiver
        flute_receiver.push(bytes(pkt))
```

Metadata

10 days ago
v1.66.0
MIT
90.5 KiB

Install

Run the following Cargo command in your project directory:

```
cargo add flute
```

Or add the following line to your Cargo.toml:

```
flute = "1.8.1"
```

Documentation

docs.rs/flute/1.8.1

Repository

github.com/ypo/flute

Owners


Yannick Poirier

Categories

- Space protocols
- Encoding
- Network programming

Report crate

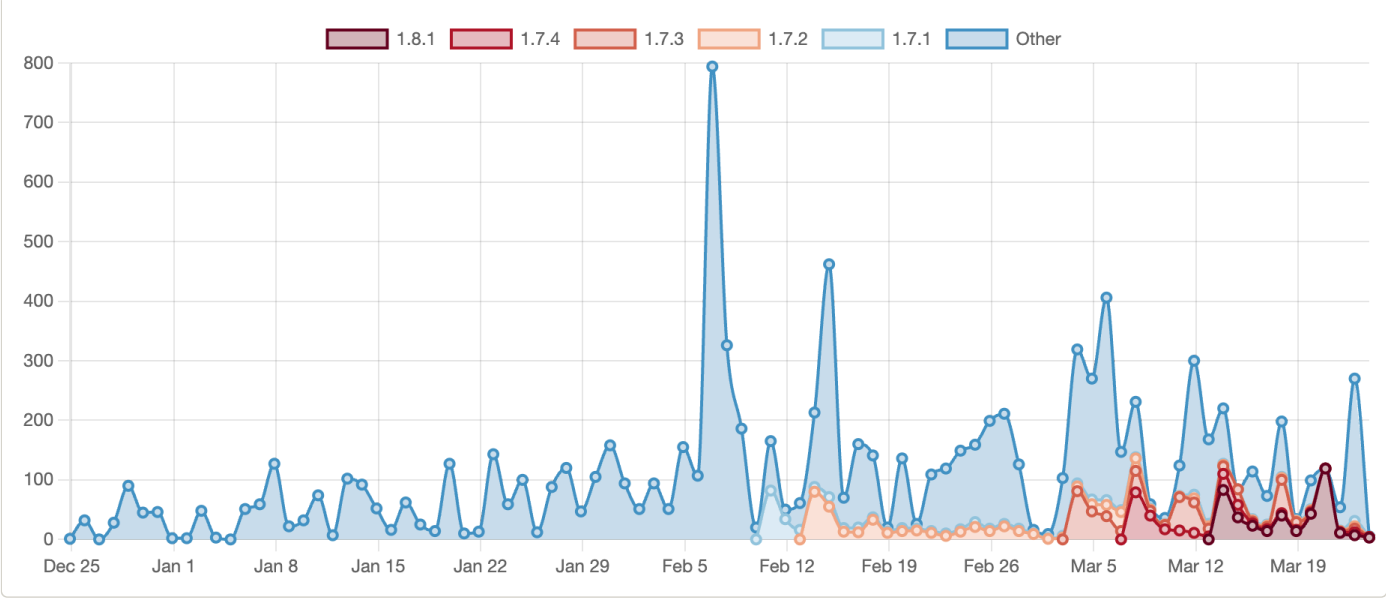
Stats Overview

35,905
Downloads all time

51
Versions published

Downloads over the last 90 days

Display as Stacked



Rust

rust-lang.org

Rust Foundation

The crates.io team

Get Help

The Cargo Book

Support

System Status

Report a bug

Policies

Usage Policy

Security

Privacy Policy

Code of Conduct

Data Access

Social

rust-lang/crates.io

#t-crates-io

@cratesiostatus